

5.4 竞赛题目举例

例题 5-8 Unix ls 命令 (Unix ls, UVa 400)

输入正整数 n 以及 n 个文件名，排序后按列优先的方式左对齐输出。假设最长文件名有 M 字符，则最右列有 M 字符，其他列都是 $M+2$ 字符。

样例输入 (略，可以由样例输出推出)

样例输出：

```
-----
Alice      Chris      Jan        Marsha     Ruben
Bobby      Cindy      Jody       Mike       Shirley
Buffy      Danny      Keith      Mr._French Sissy
Carol      Greg       Lori       Peter
```

【分析】

首先计算出 M 并算出行数，然后逐行逐列输出。代码如下：

```
#include<iostream>
#include<string>
#include<algorithm>
using namespace std;

const int maxcol = 60;
const int maxn = 100 + 5;
string filenames[maxn];

//输出字符串 s，长度不足 len 时补字符 extra
void print(const string& s, int len, char extra) {
    cout << s;
    for(int i = 0; i < len-s.length(); i++)
        cout << extra;
}

int main() {
    int n;
    while(cin >> n) {
        int M = 0;
        for(int i = 0; i < n; i++) {
            cin >> filenames[i];
            M = max(M, (int)filenames[i].length()); //STL 的 max
        }
        //计算列数 cols 和行数 rows
```

```
int cols = (maxcol - M) / (M + 2) + 1, rows = (n - 1) / cols + 1;
print("", 60, '-');
cout << "\n";
sort(filenamees, filenamees+n); //排序
for(int r = 0; r < rows; r++) {
    for(int c = 0; c < cols; c++) {
        int idx = c * rows + r;
        if(idx < n) print(filenamees[idx], c == cols-1 ? M : M+2, ' ');
    }
    cout << "\n";
}
return 0;
}
```

例题 5-9 数据库 (Database, ACM/ICPC NEERC 2009, UVa1592)

输入一个 n 行 m 列的数据库 ($1 \leq n \leq 10000$, $1 \leq m \leq 10$)，是否存在两个不同行 r_1, r_2 和两个不同列 c_1, c_2 ，使得这两行和这两列相同（即 (r_1, c_1) 和 (r_2, c_1) 相同， (r_1, c_2) 和 (r_2, c_2) 相同）。例如，对于如图 5-3 所示的数据库，第 2、3 行和第 2、3 列满足要求。

How to compete in ACM ICPC	Peter	peter@neerc.ifmo.ru
How to win ACM ICPC	Michael	michael@neerc.ifmo.ru
Notes from ACM ICPC champion	Michael	michael@neerc.ifmo.ru

图 5-3 数据库

【分析】

直接写一个四重循环枚举 r_1, r_2, c_1, c_2 可以吗？理论上可以，实际上却行不通。枚举量太大，程序会执行相当长的时间，最终获得 TLE（超时）。

解决方法是只枚举 c_1 和 c_2 ，然后从上到下扫描各行。每次碰到一个新的行 r ，把 c_1, c_2 两列的内容作为一个二元组存到一个 map 中。如果 map 的键值中已经存在这个二元组，该二元组映射到的就是所要求的 r_1 ，而当前行就是 r_2 。

这里有一个细节问题：如何表示由 c_1, c_2 两列组成的二元组？一种方法是直接用两个字符串拼成一个长字符串（中间用一个其他地方不可能出现的字符分隔），但是速度比较慢（因为在 map 中查找元素时需要进行字符串比较操作）。更值得推荐的方法是在主循环之前先做一个预处理——给所有字符串分配一个编号，则整个数据库中每个单元格都变成了整数，上述二元组就变成了两个整数。这个技巧已经在前面的例题“集合栈计算机”中用过，读者不妨再复习一下那道题目。

例题 5-10 PGA 巡回赛的奖金 (PGA Tour Prize Money, ACM/ICPC World Finals 1990, UVa207)

你的任务是为 PGA（美国职业高尔夫球协会）巡回赛计算奖金。巡回赛分为 4 轮，其中所有选手都能打前两轮（除非中途取消资格），得分相加（越少越好），前 70 名（包括

并列)晋级 (make the cut)。所有晋级选手再打两轮,前 70 名 (包括并列)有奖金。组委会事先会公布每个名次能拿的奖金比例。例如,若冠军比例是 18%,总奖金是\$1000000,则冠军奖金是\$180000。

输入保证冠军不会并列。如果第 k 名有 n 人并列,则第 $k \sim k+n-1$ 名的奖金比例相加后平均分给这 n 个人。奖金四舍五入到美分。所有业余选手不得奖金。例如,若业余选手得了第 3 名,则第 4 名会拿第 3 名的奖金比例。如果没取消资格的非业余选手小于 70 名,则剩下的奖金就不发了。

输入第一行为数据组数。每组数据前有一个空行,然后分为两部分。第一部分有 71 行 (各有一个实数),第一行为总奖金,第 $n+1$ 行为第 i 名的奖金比例。比例均保留 4 位小数,且总和为 100%。第 72 行为选手数 (最多 144),然后每行一个选手,格式为:

Player name RD1 RD2 RD3 RD4

业余选手名字后会有一“*”。犯规选手在犯规的那一轮成绩为 DQ,并且后面不再有其他成绩。但是只要没犯规,即使没有晋级,也会给出 4 轮成绩 (虽然在实际比赛中没晋级的选手只会有两个成绩)。输入保证至少有 70 个人晋级。

输入举例:

```
140
WALLY WEDGE      70 70 70 70
SANDY LIE        80 DQ
SID SHANKER*     90 99 62 61
...
JIMMY ABLE       69 73 80 DQ
```

输出应包含所有晋级到后半段 (make the cut) 的选手。输出信息包括:选手名字、排名、各轮得分、总得分以及奖金数。没有得奖则不输出,若有奖金,即使奖金是\$0.00 也要输出,保留两位小数)。如果此名次至少有两个人获得奖金,应在名次后面加“T”。犯规选手列在最后,总得分为 DQ,名次为空。如果有并列,则先按轮数排序,然后按各轮得分之和排序,最后按名字排序。两组数据的输出之间用一个空格隔开。

输出举例:

Player Name	Place	RD1	RD2	RD3	RD4	TOTAL	Money Won
WALLY WEDGE	1	70	70	70	70	280	\$180000.00
HENRY HACKER	2T	77	70	70	70	287	\$88000.00
TOMMY TWO IRON	2T	71	72	72	72	287	\$88000.00
BEN BIRDIE	4	70	74	72	72	288	\$48000.00
NORMAN NIBLICK*	4	72	72	72	72	288	
...							
LEE THREE WINES	71	99	99	99	98	395	\$2000.00
JOHNY MELAVO	72	99	99	99	99	396	
JIMMY ABLE		69	73	80		DQ	
EDDIE EAGLE		71	71			DQ	

【分析】

不难发现，第一个步骤是选出晋级选手，这涉及对所有选手“前两轮总得分”进行排序。接下来计算4轮总分，然后再排序一次，最后对排序结果依次输出。

输出过程不能大意：犯规选手要单独处理；在输出一行之前要先看看有没有并列的情况，如有则要一并处理（包括计算奖金平分情况）。本题没有技术上的难度，但比较考验选手的代码组织能力和对细节的处理，推荐读者一试。

例题 5-11 邮件传输代理的交互 (The Letter Carrier's Rounds, ACM/ICPC World Finals 1999, UVa814)

本题的任务为模拟发送邮件时 MTA（邮件传输代理）之间的交互。所谓 MTA，就是 email 地址格式 `user@mtaname` 的“后面部分”。当某人从 `user1@mta1` 发送给另一个人 `user2@mta2` 时，这两个 MTA 将会通信。如果两个收件人属于同一个 MTA，发送者的 MTA 只需与这个 MTA 通信一次就可以把邮件发送给这两个人。

输入每个 MTA 里的用户列表，对于每个发送请求（输入发送者和接收者列表），按顺序输出所有 MTA 之间的 SMTP（简单邮件协议）交互。协议细节参见原题。

发送人 MTA 连接收件人 MTA 的顺序应该与在输入中第一次出现的顺序一致。例如，若发件人是 `Hamdy@Cairo`，收件人列表为 `Conrado@MexicoCity`、`Shariff@SanFrancisco`、`Lisa@MexicoCity`，则 Cairo 应当依次连接 MexicoCity 和 SanFrancisco。

如果连接某个 MTA 之后发现所有收件人都不存在，则不应该发送 DATA。所有用户名均由不超过 15 个字母和数字组成。

【分析】

本题的关键是理清各个名词之间的逻辑关系以及把要做的事情分成几个步骤。首先是输入过程，把每个 MTA 里的用户列表保存下来。一种方法是用一个 `map<string, vector<string>>`，其中键是 MTA 名称，值是用户名列表。一个更简单的方法是用一个 `set<string>`，值就是邮件地址。

对于每个请求，首先读入发件人，分离出 MTA 和用户名，然后读入所有收件人，根据 MTA 出现的顺序进行保存，并且去掉重复。接下来读入邮件正文，最后按顺序依次连接每个 MTA，检查并输出每个收件人是否存在，如果至少有一个存在，则输出邮件正文。

本题的整个解决过程并不复杂，对于初学者来说是个不错的基础练习。参考代码如下：

```
#include<iostream>
#include<string>
#include<vector>
#include<set>
#include<map>
using namespace std;

void parse_address(const string& s, string& user, string& mta) {
    int k = s.find('@');
    user = s.substr(0, k);
    mta = s.substr(k+1);
}
```

```

int main() {
    int k;
    string s, t, user1, mta1, user2, mta2;
    set<string> addr;

    //输入所有 MTA, 转化为地址列表
    while(cin >> s && s != "") {
        cin >> s >> k;
        while(k--) { cin >> t; addr.insert(t + "@" + s); }
    }

    while(cin >> s && s != "") {
        parse_address(s, user1, mta1); //处理发件人地址

        vector<string> mta; //所有需要连接的 mta, 按照输入顺序
        map<string, vector<string> > dest; //每个 MTA 需要发送的用户
        set<string> vis;
        while(cin >> t && t != "") {
            parse_address(t, user2, mta2); //处理收件人地址
            if(vis.count(t)) continue; //重复的收件人
            vis.insert(t);
            if(!dest.count(mta2)) { mta.push_back(mta2); dest[mta2] = vector<string>(); }
            dest[mta2].push_back(t);
        }
        getline(cin, t); //把 "*" 这一行的回车吃掉

        //输入邮件正文
        string data;
        while(getline(cin, t) && t[0] != '*') data += " " + t + "\n";

        for(int i = 0; i < mta.size(); i++) {
            string mta2 = mta[i];
            vector<string> users = dest[mta2];
            cout << "Connection between " << mta1 << " and " << mta2 << endl;
            cout << " HELO " << mta1 << "\n"; cout << " 250\n";
            cout << " MAIL FROM:<" << s << ">\n"; cout << " 250\n";
            bool ok = false;
            for(int i = 0; i < users.size(); i++) {
                cout << " RCPT TO:<" << users[i] << ">\n";
                if(addr.count(users[i])) { ok = true; cout << " 250\n"; }
                else cout << " 550\n";
            }
        }
    }
}

```

```

if(ok) {
    cout << " DATA\n"; cout << " 354\n";
    cout << data;
    cout << ".\n"; cout << " 250\n";
}
cout << " QUIT\n"; cout << " 221\n";
}
}
return 0;
}

```

例题 5-12 城市正视图 (Urban Elevations, ACM/ICPC World Finals 1992, UVa221)

如图 5-4 所示, 有 n ($n \leq 100$) 个建筑物。左侧是俯视图 (左上角为建筑物编号, 右下角为高度), 右侧是从南向北看的正视图。

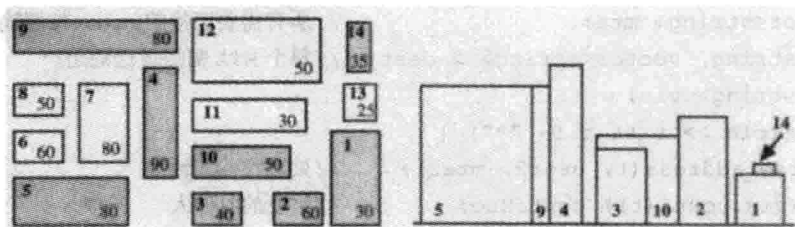


图 5-4 建筑俯视图与正视图

输入每个建筑物左下角坐标 (即 x 、 y 坐标的最小值)、宽度 (即 x 方向的长度)、深度 (即 y 方向的长度) 和高度 (以上数据均为实数), 输出正视图中能看到的所有建筑物, 按照左下角 x 坐标从小到大进行排序。左下角 x 坐标相同时, 按 y 坐标从小到大排序。

输入保证不同的 x 坐标不会很接近 (即任意两个 x 坐标要么完全相同, 要么差别足够大, 不会引起精度问题)。

【分析】

注意到建筑物的可见性等价于南墙的可见性, 可以在输入之后直接忽略“深度”这个参数。接下来把建筑物按照输出顺序排序, 然后依次判断每个建筑物是否可见。

判断可见性看上去比较麻烦, 因为一个建筑物可能只有部分可见, 无法枚举所有 x 坐标, 来查看这个建筑物在该处是否可见, 因为 x 坐标有无穷多个。解决方法有很多种, 最常见的是离散化, 即把无穷变为有限。

具体方法是: 把所有 x 坐标排序去重, 则任意两个相邻 x 坐标形成的区间具有相同属性, 一个区间要么完全可见, 要么完全不可见。这样, 只需在这个区间里任选一个点 (例如中点), 就能判断出一个建筑物是否在整个区间内可见。如何判断一个建筑物是否在某个 x 坐标处可见呢? 首先, 建筑物的坐标中必须包含这个 x 坐标, 其次, 建筑物南边不能有另外一个建筑物也包含这个 x 坐标, 并且不比它矮。

```

#include<cstdio>
#include<algorithm>

```

```

using namespace std;

const int maxn = 100 + 5;

struct Building {
    int id;
    double x, y, w, d, h;
    bool operator < (const Building& rhs) const {
        return x < rhs.x || (x == rhs.x && y < rhs.y);
    }
} b[maxn];

int n;
double x[maxn*2];

bool cover(int i, double mx) {
    return b[i].x <= mx && b[i].x+b[i].w >= mx;
}

//判断建筑物 i 在 x=mx 处是否可见
bool visible(int i, double mx) {
    if(!cover(i, mx)) return false;
    for(int k = 0; k < n; k++)
        if(b[k].y < b[i].y && b[k].h >= b[i].h && cover(k, mx)) return false;
    return true;
}

int main() {
    int kase = 0;
    while(scanf("%d", &n) == 1 && n) {
        for(int i = 0; i < n; i++) {
            scanf("%lf%lf%lf%lf%lf", &b[i].x, &b[i].y, &b[i].w, &b[i].d, &b[i].h);
            x[i*2] = b[i].x; x[i*2+1] = b[i].x + b[i].w;
            b[i].id = i+1;
        }
        sort(b, b+n);
        sort(x, x+n*2);
        int m = unique(x, x+n*2) - x; //x坐标排序后去重, 得到 m 个坐标

        if(kase++) printf("\n");
        printf("For map #%d, the visible buildings are numbered as follows:\n%d",
            kase, b[0].id);
    }
}

```